

---

---

# Performance and Memory Cost Comparison of AES, ASCON, and TinyJAMBU on Edge Devices

密碼學期末專題  
組員：吳柏慶(R13921A24)

---

---

# OUTLINE

1. Introduction
2. Algorithm Specifications
3. Experiment
4. Discussion
5. Conclusion and Future Work

# Introduction

- The IoT infrastructure has reached unprecedented scale, with an estimated 30–50 billion connected **edge devices** operating across smart cities, industrial automation systems, healthcare monitoring networks, and environmental sensing grids.
- These edge devices typically face severe **resource constraints**: only 2 KB of SRAM and 32 KB of Flash.
- The current de facto standard for network security, the **AES-128-GCM**, was designed for high-end computing environments equipped with dedicated **hardware acceleration** instruction sets.

# Introduction

- NIST initiated the **Lightweight Cryptography (LWC)** Standardization Process in 2015.
- The standard, published as NIST SP 800-232 in August 2025, specifies **Ascon-AEAD128** as the primary authenticated encryption algorithm for constrained devices.
- **TinyJAMBU-128 v2**—the extreme memory-optimized finalist
- We compare the Performance and Cost on encryption algorithm **AES, ASCON, and TinyJAMBU**.

# Algorithm Specifications

- ASCON Permutations

$$p = p_L \circ p_S \circ p_C$$

- Internal State (320 bits)

$$\mathcal{S} = S_0 \parallel S_1 \parallel S_2 \parallel S_3 \parallel S_4$$

- Constant-Addition Layer  $p_C$

$$\text{Ascon-}p[\text{rnd}] \quad c_i = \text{const}_{16-\text{rnd}+i}$$

$$S_2 = S_2 \oplus c_i$$

# Algorithm Specifications

- Substitution Layer  $p_S$

$$(s_{(0,j)}, s_{(1,j)}, \dots, s_{(4,j)}) = \text{SBox}(s_{(0,j)}, s_{(1,j)}, \dots, s_{(4,j)})$$

- Linear Diffusion Layer  $p_L$

$$\Sigma_0(S_0) = S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28)$$

$$\Sigma_1(S_1) = S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39)$$

$$\Sigma_2(S_2) = S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6)$$

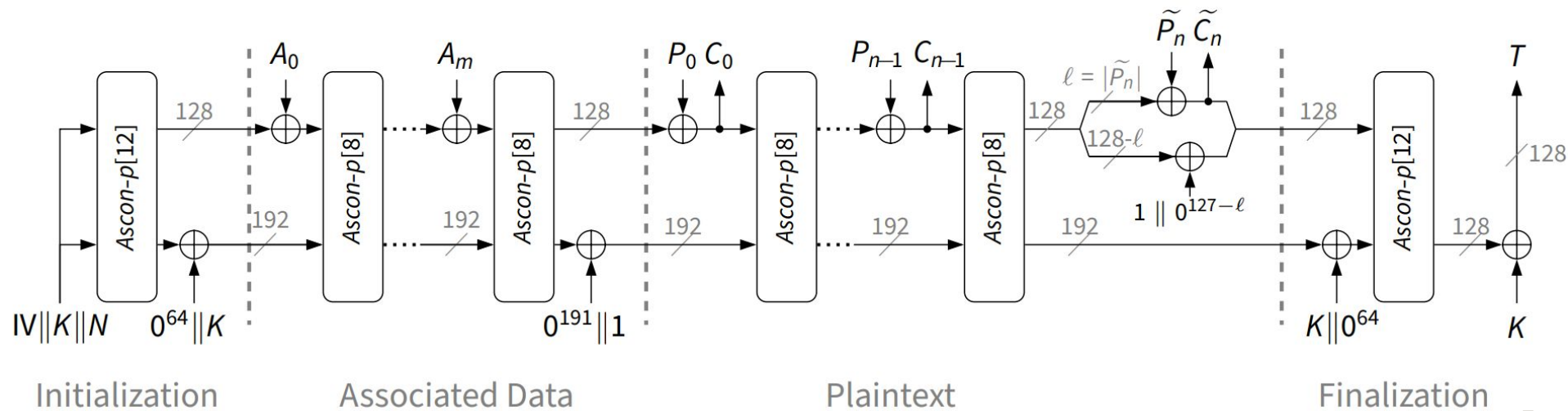
$$\Sigma_3(S_3) = S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17)$$

$$\Sigma_4(S_4) = S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41)$$

# Algorithm Specifications

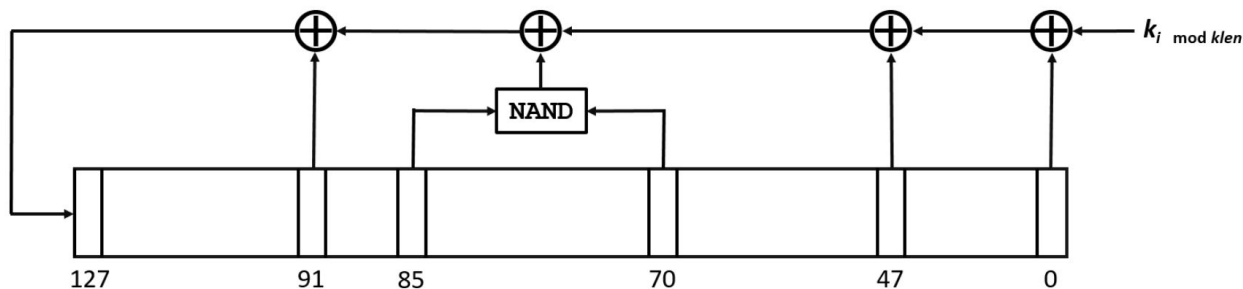
- Ascon-AEAD128: 128-bit secret key  $K$ , a 128-bit nonce  $N$ , associated data  $A$ , and plaintext  $P$  as inputs and outputs ciphertext  $C$  and authentication tag  $T$   

$$\text{Ascon-AEAD128. enc}(K, N, A, P) = (C, T)$$



# Algorithm Specifications

- TinyJAMBU-128: 128-bit nonlinear feedback shift register(NLFSR)
- permutation  $P_n$  consists of n rounds



- Key Setup: Set the 128-bit state  $S$  as 0, and update the state using  $P_{1024}$
- FrameBits: Three-bit FrameBits to recognize the current data type



# Algorithm Specifications

- Nonce Setup: 128-bit nonlinear feedback shift register(NLFSR)

for  $i$  from 0 to 2:

$$s_{\{36...38\}} = s_{\{36...38\}} \oplus FrameBits_{\{0...2\}}$$

Update the state using  $P_{640}$

$$s_{\{96...127\}} = s_{\{96...127\}} \oplus nonce_{\{32i...32i+31\}}$$

end for

# Algorithm Specifications

- TinyJAMBU Encryption:

for  $i$  from 0 to  $\lfloor mlen/32 \rfloor$ :

$$s_{\{36 \dots 38\}} = s_{\{36 \dots 38\}} \oplus FrameBits_{\{0 \dots 2\}}$$

Update the state using  $P_{1024}$

$$s_{\{96 \dots 127\}} = s_{\{96 \dots 127\}} \oplus m_{\{32i \dots 32i+31\}}$$

$$c_{\{32i \dots 32i+31\}} = s_{\{64 \dots 95\}} \oplus m_{\{32i \dots 32i+31\}}$$

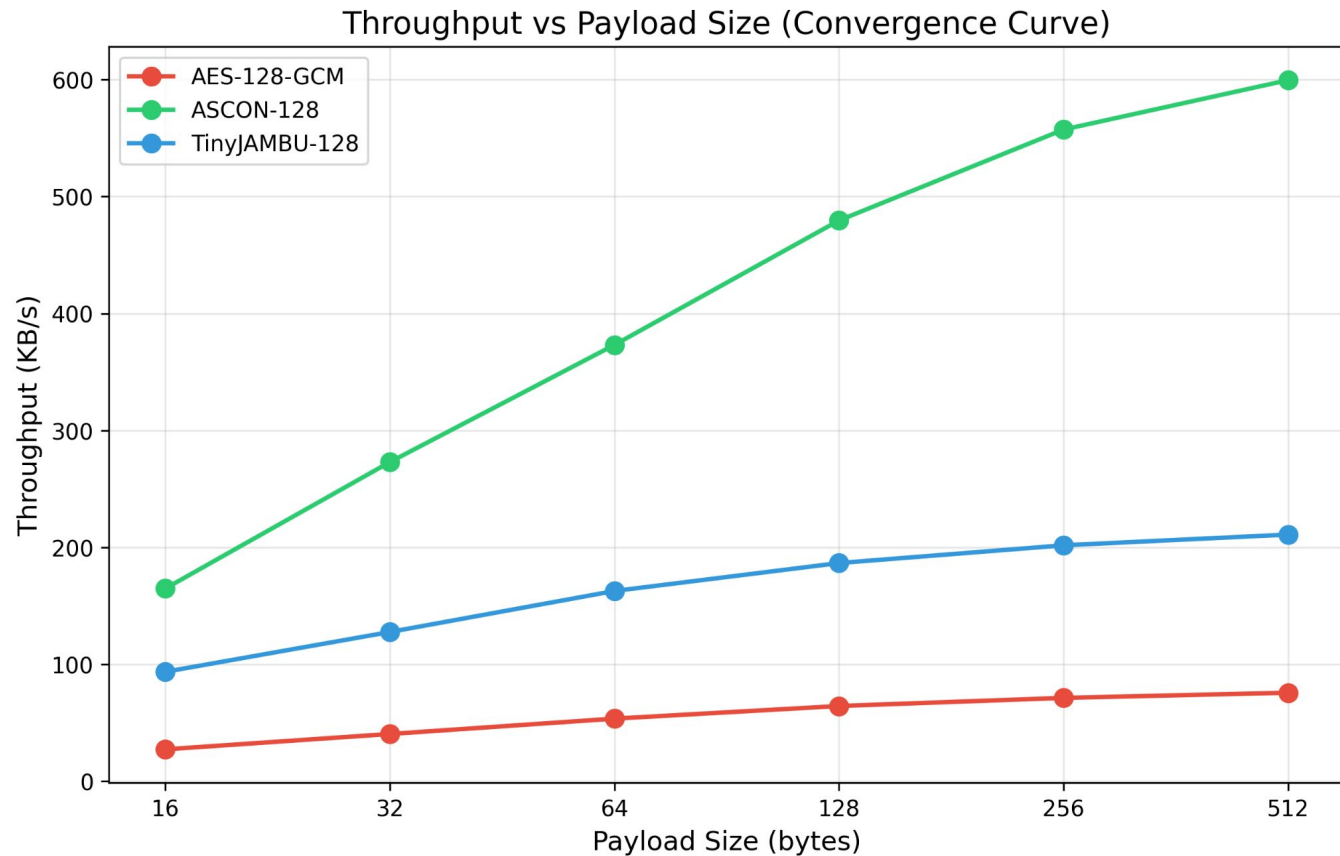
end for

# Experiment

- Implementation:
  - AES-128-GCM: Pure Python. GCM mode. w/o hardware acceleration.
  - Ascon-AEAD128: Pure Python, NIST SP 800-232 little-endian.
  - TinyJAMBU-128 v2: Pure Python, FBK\_32 optimized. Uses  $4 \times 32$ -bit word state representation with P1024/P640 keyed permutations.
- Measuremen:
  - 20 warmup iterations; 200 measurement iterations
  - Initialization phase timed separately

# Experiment

## 1. Throughput



# Experiment

## 2. Initialization Cost Analysis

| $ P $ (B) | AES-128-GCM | Ascon-AEAD128 | TinyJAMBU-128 v2 |
|-----------|-------------|---------------|------------------|
| 16        | 31.1%       | 25.2%         | 28.6%            |
| 32        | 22.8%       | 19.9%         | 19.2%            |
| 64        | 15.3%       | 14.0%         | 13.0%            |
| 128       | 9.2%        | 8.8%          | 7.2%             |
| 256       | 5.0%        | 5.4%          | 3.9%             |
| 512       | 2.7%        | 2.8%          | 2.0%             |

# Experiment

## 3. Memory Footprint

| Metric                | AES-128-GCM  | Ascon-AEAD128 | TinyJAMBU-128 v2 |
|-----------------------|--------------|---------------|------------------|
| Internal state (bits) | 512+         | 320           | 128              |
| Lookup tables (bytes) | 8,192        | 0             | 0                |
| Est. ROM (bytes)      | 6,000–12,000 | 1,000–2,000   | 500–1,000        |
| Est. RAM (bytes)      | 60–100       | 40–80         | 16–32            |

# Experiment

## 4. Code Size

| Algorithm          | File Size (B) | SLOC |
|--------------------|---------------|------|
| AES-128-GCM (pure) | 12,147        | 250  |
| Ascon-AEAD128      | 7,761         | 184  |
| TinyJAMBU-128 v2   | 8,885         | 170  |

# Experiment

## 5. Energy Consumption

Table 5: Estimated  $E_{\text{enc}}$  ( $\mu\text{J}$ ) on Cortex-M3 (84 MHz, 33.6 mW)

| $ P $ (B) | AES-128-GCM | Ascon-AEAD128 | TinyJAMBU-128 v2 |
|-----------|-------------|---------------|------------------|
| 16        | 19.906      | 3.285         | 5.821            |
| 32        | 26.207      | 4.006         | 8.121            |
| 64        | 39.106      | 5.612         | 13.038           |
| 128       | 65.222      | 8.578         | 22.502           |
| 256       | 118.135     | 15.244        | 42.120           |
| 512       | 222.517     | 28.265        | 79.289           |



# Discussion

- AES-128-GCM with hardware acceleration would yield extremely high throughput ( $> 100,000$  KB/s)
- However, on edge devices w/o hardware acceleration and with short packets (16–128 B), ASCON beats AES on throughput, Initialization Cost, Memory, Code Size, and Energy Consumption.
- Among 3 cryptography algorithms, ASCON performs the best; hence, it wins the NIST final LWC standard.
- TinyJAMBU takes the least RAM among all competitors, which may become the best solution under the extremely constrained edge devices ( $< 4$  KB SRAM).

# Conclusion and Future Work

- Platform: x86\_64 Python does not replicate 8/32-bit MCU register constraints. Native C benchmarks on physical MCUs are the next step.
- Attack: Side-channel or other attacks are not evaluated.